灵巧手工作台纸箱折叠技能 Demo 项目文档

总体架构、接口规范与实施方案

字段	内容
文档版本	正式版
编制日期	2026-04-10
适用范围	项目经理、AI 算法工程师、SDK 工程师、应用开发工程师、联调人员
项目目标	在固定工位上完成未折叠纸箱到成品箱子的单次端到端折叠演示
总体方案	单进程主控 + 配置驱动 StepGraph + 结构化视觉观测 + 原子动作执行器

文档定位

- 本文件直接作为项目执行文档使用，用于统一架构设计、接口边界、数据结构、实施方式与验收标准。
- 本文档默认场景为单工位、单箱型、单次闭环演示；不覆盖多纸箱连续作业与复杂力控策略。
- 核心控制闭环为 observe → decide → act → verify，动作执行成功不等于步骤成功，必须经过视觉后置条件验证。

关键交付节点（已锁定）

日期	阶段目标	验收要求
2026-04-20（周一）前	完整 Demo 开发、集成、单次端到端测试完成	完整闭环跑通，日志与回放可用
2026-04-21（周二）	全天稳定性调试与优化	连续运行稳定，无阻断性故障
2026-04-22（周三）	支持节目录制	演示全流程无故障、可复现、可讲解

1. 项目概述

本项目面向灵巧手工作台场景，目标是在固定桌面、固定俯视深度相机、固定纸箱规格的条件下，实现“操作员放置纸箱—视觉判断当前状态—系统调度原子动作—机械臂与灵巧手执行—视觉复核并推进下一步”的端到端演示。项目以稳定、可联调、可录制、可回放为优先目标，强调结构清晰、接口明确与配置可维护。

1.1 项目目标

- 完成未折叠纸箱到成品箱子的单次全流程折叠演示。
- 验证机械臂、灵巧手、RGB-D 相机、视觉模型与顶层控制之间的协同能力。
- 建立可复用的项目骨架，为后续多技能任务提供架构基础、接口规范与调试方法。

1.2 项目范围与非范围

类别	内容
项目范围	固定工位；固定俯视相机；单箱型；单次折叠闭环；视觉判断 + 预定义原子动作；日志与回放能力。
非范围	多纸箱连续作业；复杂力控闭环；动态相机位姿变化；跨多工位调度；在线任务学习。

1.3 成功标准

维度	判定标准
功能闭环	系统能够从纸箱放置开始，完成全部折叠步骤并输出完成状态。
结构正确	感知、控制、执行三层边界清晰，Step / Action / Fact 不混用。
验证闭环	每个关键动作后均进行视觉后置条件验证，不以轨迹完成代替步骤完成。
工程可用	具备配置文件、错误码、运行日志、回放素材与基础恢复策略。
联调可控	接口强类型、字段命名统一、异常可定位，支持多人并行开发。

2. 总体架构设计

系统总体架构采用“单进程主控 + 配置驱动 StepGraph + 结构化视觉观测 + 原子动作执行器”的设计。该架构优先解决本项目最重要的工程问题：在既定交付时间内，以尽量少的系统复杂度达成稳定演示。

2.1 设计原则

原则	说明
单一主控	以 Orchestrator 作为运行态单一真相源，统一维护当前步骤、重试计数、上一次动作与任务状态。
分层解耦	感知层输出 Observation 和 Facts；控制层输出 Decision；执行层只执行 action_id，不自行决定流程。
配置驱动	步骤转移、动作映射、参数与恢复策略均以配置文件管理，避免大量 if/else 硬编码。
验证闭环	动作执行结果仅表示动作是否跑完，步骤推进必须由视觉验证后置条件。
回放优先	每次感知、决策、执行均记录结构化事件，便于联调、复盘与录制前排障。

2.2 模块职责划分

模块	主要职责	输入	输出
CameraAdapter	采集 RGB-D、裁剪 ROI、附带标定信息	采集请求 / 相机配置	FramePacket
PerceptionService	步骤候选识别、结构化事实抽取、质量评估	FramePacket + 轻量上下文	PerceptionResult
Orchestrator	维护 RunContext、调度模块、写入日志	PerceptionResult / ExecutionResult	更新后的运行态
StepGraph Planner	根据当前步骤、事实、重试状态给出动作或恢复决策	RunContext + PerceptionResult + StepGraph	DecisionResult
ActionExecutor	调用机械臂和灵巧手 SDK 执行原子动作	ExecutionRequest	ExecutionResult
Config / Log / Replay	保存配置、运行事件、图像素材和回放索引	各模块事件	调试与审计支撑

2.3 总体架构图

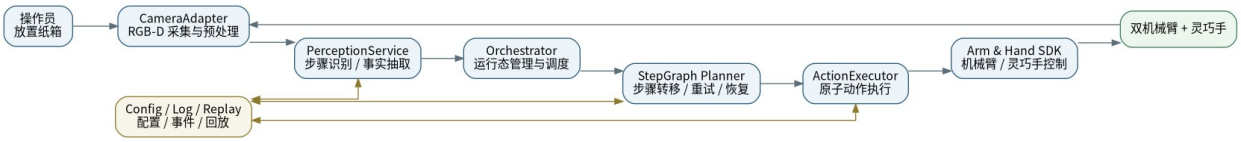

图 1 总体架构图

控制路径自左向右流动，执行后通过相机重新观测形成反馈闭环；配置、日志与回放构成支撑层。

2.4 端到端控制闭环

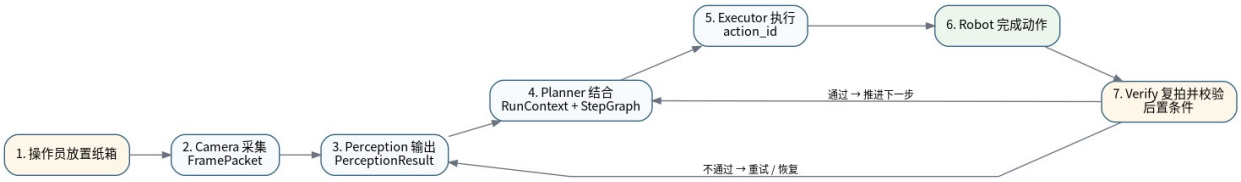

图 2 端到端控制闭环图

系统以 Observe → Decide → Act → Verify 为最小执行单元；验证不通过时，不直接推进步骤，而是进入重试或恢复。

2.5 运行状态机

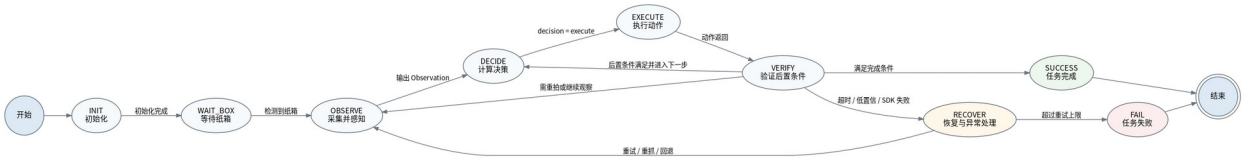

图 3 运行状态机图

执行成功不必然导致状态推进；只有后置条件满足时，状态机才会进入下一逻辑步骤。

3. 流程编排与控制逻辑

3.1 Step / Action / Fact 三类对象分层

对象	定义	示例	设计要求
Step	流程逻辑中的步骤节点	S20_fold_left_flap	表示“当前流程走到哪里”，不直接等同于物理动作。
Action	执行层可调用的原子动作	A20_fold_left_flap_v1	表示“机器人本次要做什么”，允许同一步骤映射多个动作版本。
Fact	视觉可消费的结构化事实	left_flap = closed	表示“当前世界状态是什么”，作为步骤推进和恢复判断依据。

核心约束

- Perception 不直接输出 next_action，避免视觉模块越权控制执行逻辑。
- ActionExecutor 不接受裸 step_id 作为唯一输入，而是接受完整的 ExecutionRequest。
- 任何“自然语言说明”都不能直接驱动控制逻辑，必须先落到结构化 facts。

3.2 控制闭环定义

- 采集：CameraAdapter 输出包含 RGB、Depth、时间戳、相机版本与工作空间版本的 FramePacket。
- 感知：PerceptionService 输出步骤候选、事实集合、质量评分与是否需要重拍。
- 决策：Planner 基于 RunContext、StepGraph 与 PerceptionResult 生成 execute / retry / recover / finish 等决策。
- 执行：Executor 依据 action_id、参数与超时时间执行动作，并返回详细执行结果。
- 验证：再次采集图像并核验后置条件；通过则推进步骤，否则进入重试或恢复。
- 记录：每个环节都写入结构化事件，形成完整的回放链路。

3.3 StepGraph 转移规则

规则类型	作用	说明
preconditions	动作前检查	用于保证动作在合理前提下执行，例如 box_present = true。
postconditions	动作后验证	用于决定步骤是否真正完成，例如 left_flap = closed。
on_success	成功后转移	后置条件满足时进入下一步骤。
on_fail	失败后恢复	执行失败或后置条件不满足时进入恢复动作或重试逻辑。
max_retry	重试上限	用于防止系统无限循环。

3.4 异常与恢复策略

异常类型	判据	默认处理
低置信度识别	top1_score 低于阈值，或 top1 与 top2 分数差过小	先 retake；连续失败达到阈值后转 recover。
图像质量差	quality_score 低、遮挡严重、深	重新采图，必要时提示人工调整纸

	度异常	箱位置。
动作执行失败	SDK 报错、轨迹失败、抓取失败	记录错误码并进入对应恢复动作。
后置条件未满足	动作完成但视觉未确认步骤达成	当前步骤不推进，允许 retry 或回退。
不可恢复状态	连续恢复失败或状态冲突	停止任务并输出 abort / fail。

4. 数据结构与接口规范

本节定义运行时核心对象、接口签名、枚举与错误码。所有跨模块接口均建议以强类型对象承载，推荐使用 dataclass 或 Pydantic 模型统一生成校验逻辑与序列化结果。

4.1 命名与 ID 约定

字段	含义	示例
run_id	一次完整 Demo 运行实例 ID	run_20260410_001
frame_id	单帧图像 ID	frame_000123
step_id	逻辑步骤 ID	S20_fold_left_flap
action_id	执行动作 ID	A20_fold_left_flap_v1
box_spec_id	纸箱规格 / 工艺版本	carton_demo_v1
workspace_id	工位版本	green_table_v1
event_id	运行事件 ID	event_001245
fact_key	视觉事实键	left_flap / body_opened

4.2 核心对象关系图

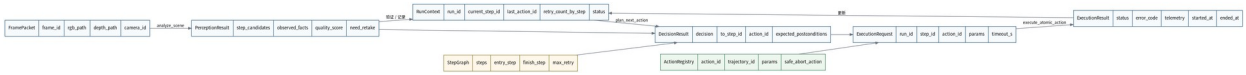

图 4 核心对象关系图

FramePacket 驱动感知，DecisionResult 驱动执行，ExecutionResult 与 PerceptionResult 共同回写 RunContext。

4.3 标准接口定义

接口	职责	输入	输出
capture_frame(req)	采集一帧带元数据的图像	CaptureRequest	FramePacket

analyze_scene(frame, ctx_lite)	输出步骤候选与结构化事实	FramePacket, RunContextLite	PerceptionResult
plan_next_action(ctx, obs)	输出动作、重试或恢复决策	RunContext, PerceptionResult	DecisionResult
execute_atomic_action (req)	执行原子动作	ExecutionRequest	ExecutionResult
record_event(event)	记录结构化运行事件	RuntimeEvent	None

4.4 核心对象详细定义

4.4.1 RunContext

RunContext 是顶层控制器维护的唯一可信运行态，不允许各模块维护独立的主状态。

字段	类型	说明
schema_version	string	数据结构版本号。
run_id	string	任务运行实例 ID。
task_type	string	任务类型，例如 carton_folding。
box_spec_id	string	当前纸箱规格。
workspace_id	string	当前工位版本。
status	enum	running / success / fail 等运行态。
current_step_id	string	当前逻辑步骤。
last_action_id	string	最近一次已执行动作。
retry_count_by_step	map	每个步骤的重试次数。
started_at / updated_at	datetime	运行开始与更新时间。

```
{
  "schema_version": "1.0.0",
  "run_id": "run_20260410_001",
  "task_type": "carton_folding",
  "box_spec_id": "carton_demo_v1",
  "workspace_id": "green_table_v1",
  "status": "running",
  "current_step_id": "S20_fold_left_flap",
  "last_action_id": "A10_open_box_body_v1",
  "retry_count_by_step": {"S20_fold_left_flap": 1}
}
```

4.4.2 FramePacket

FramePacket 为感知输入对象。跨模块传递时优先传路径与元数据，不建议以 base64 直接传整帧图像。

字段	类型	说明
frame_id	string	图像 ID。
ts	datetime	采集时间戳。
camera_id	string	相机标识。
rgb_path	string	RGB 图像路径。
depth_path	string	Depth 图像路径。
intrinsics_version	string	内参版本。
extrinsics_version	string	外参版本。
workspace_id	string	工作空间版本。

```
{
  "frame_id": "frame_000123",
  "ts": "2026-04-10T10:00:01.123+08:00",
  "camera_id": "top_depth_cam",
  "rgb_path": "artifacts/run_20260410_001/frame_000123_rgb.jpg",
  "depth_path": "artifacts/run_20260410_001/frame_000123_depth.png",
  "intrinsics_version": "cam_intr_v3",
  "extrinsics_version": "cam_extr_v2",
  "workspace_id": "green_table_v1"
}
```

4.4.3 PerceptionResult

PerceptionResult 负责把视觉输出约束为控制层可消费的结构化信息，不允许仅输出自然语言判断。

字段	类型	说明
frame_id	string	对应图像 ID。
scene_state	enum	当前场景粗状态，例如 box_present。
step_candidates	list	候选步骤及其分数，按分数降序排列。
observed_facts	map	结构化事实集合。
quality_score	float	图像质量评分。
need_retake	bool	是否建议重拍。
need_human_check	bool	是否建议人工检查。

debug	object	仅供调试，不参与控制逻辑。

```
{
  "frame_id": "frame_000123",
  "scene_state": "box_present",
  "step_candidates": [
    {"step_id": "S20_fold_left_flap", "score": 0.91},
    {"step_id": "S10_open_box_body", "score": 0.06}
  ],
  "observed_facts": {
    "box_present": true,
    "body_opened": true,
    "left_flap": "open",
    "right_flap": "open",
    "front_flap": "unknown",
    "rear_flap": "unknown",
    "box_pose_stable": true
  },
  "quality_score": 0.94,
  "need_retake": false,
  "need_human_check": false
}
```

4.4.4 DecisionResult

DecisionResult 是唯一允许输出 action_id 的对象，用于连接控制层与执行层。

字段	类型	说明
decision	enum	execute / retake / retry / recover / finish / abort。
from_step_id	string	当前步骤。
to_step_id	string	目标步骤。
action_id	string	待执行动作；非 execute 时可为空。
reason_code	string	做出该决策的原因。
timeout_s	int	动作超时时间。
expected_postconditions	map	动作完成后需要验证的后置条件。

```
{
  "decision": "execute",
  "from_step_id": "S10_open_box_body",
  "to_step_id": "S20_fold_left_flap",
  "action_id": "A20_fold_left_flap_v1",
  "reason_code": "POSTCONDITION_MET",
  "timeout_s": 8,
  "expected_postconditions": {"left_flap": "closed"}
}
```

4.4.5 ExecutionRequest / ExecutionResult

执行层接口不应只接收 step_id，必须接收完整上下文，以支持动作参数、超时、恢复动作与验后条件。

对象	关键字段	说明
ExecutionRequest	run_id, step_id, action_id, params, timeout_s, expected_postconditions	描述一次可执行动作请求。
ExecutionResult	status, success, error_code, error_message, telemetry, started_at, ended_at	描述动作执行结果与可追踪信息。

```
{
  "request_id": "req_exec_001",
  "run_id": "run_20260410_001",
  "step_id": "S20_fold_left_flap",
  "action_id": "A20_fold_left_flap_v1",
  "params": {"speed_scale": 0.6, "approach_offset_mm": 8},
  "timeout_s": 8,
  "expected_postconditions": {"left_flap": "closed"}
}

{
  "request_id": "req_exec_001",
  "run_id": "run_20260410_001",
  "action_id": "A20_fold_left_flap_v1",
  "status": "succeeded",
  "success": true,
  "error_code": "OK",
  "telemetry": {"arm": "left", "hand_profile": "pinch"}
}
```

4.5 枚举与错误码

枚举	取值	说明
flap_state	open / half / closed / unknown	统一纸箱折页状态枚举。
decision	execute / retake / retry / recover / finish / abort	控制层输出决策。
execution_status	queued / running / succeeded / failed / timeout / aborted	动作执行状态。
scene_state	box_absent / box_present / task_done / unknown	场景粗分类。

标准错误码建议

错误码	说明

OK	正常完成。
CAMERA_TIMEOUT	相机采集超时。
LOW_CONFIDENCE	视觉识别低置信。
ACTION_TIMEOUT	动作执行超时。
GRASP_FAILED	抓取或接触失败。
SDK_ERROR	机械臂或灵巧手 SDK 报错。
POSTCHECK_FAILED	后置条件验证失败。
UNRECOVERABLE_STATE	进入不可恢复状态。

5. 配置规范与代码组织

5.1 StepGraph 配置示例

不建议将流程写成大段 if/else；推荐以 StepGraph 配置文件管理流程、前置条件、后置条件、超时与恢复策略。

```
schema_version: 1.0.0
task_type: carton_folding
box_spec_id: carton_demo_v1
entry_step: S00_wait_box
finish_step: S99_done

steps:
  S00_wait_box:
    description: wait for carton
    success_if:
      - fact: box_present
        eq: true
    next_step: S10_open_box_body

  S10_open_box_body:
    action_id: A10_open_box_body_v1
    preconditions:
      - fact: box_present
        eq: true
    postconditions:
      - fact: body_opened
        eq: true
    timeout_s: 10
    max_retry: 2
    on_fail: S10R_regrasp

  S20_fold_left_flap:
    action_id: A20_fold_left_flap_v1
    preconditions:
      - fact: body_opened
        eq: true
    postconditions:
      - fact: left_flap
        eq: closed
    timeout_s: 8
    max_retry: 1
    on_success: S30_fold_right_flap
```

5.2 ActionRegistry 配置示例

ActionRegistry 负责动作细节，包括执行器、轨迹编号、动作参数与安全回退动作。

```
actions:
  A10_open_box_body_v1:
    executor: dual_arm_hand
    arm: both
    hand_shape: support_open
    trajectory_id: traj_open_box_01
    params:
      speed_scale: 0.5
    safe_abort_action: A00_home_safe_v1

  A20_fold_left_flap_v1:
    executor: dual_arm_hand
    arm: left
    hand_shape: pinch
    trajectory_id: traj_fold_left_01
    params:
      speed_scale: 0.6
      approach_offset_mm: 8
    safe_abort_action: A00_home_safe_v1
```

5.3 其他配置文件建议

配置类别	建议文件	说明
步骤图配置	configs/step_graph/carton_demo_v1.yaml	定义步骤关系与恢复逻辑。
动作注册表	configs/actions/carton_demo_v1.yaml	定义动作实现与参数。
视觉提示词	configs/prompts/carton_judge_v1.yaml	定义感知侧 prompt 与输出约束。
工作空间配置	configs/workspace/green_table_v1.yaml	定义 ROI、桌面坐标、屏蔽区域。
相机标定配置	configs/camera/top_depth_cam_v*.yaml	定义相机内外参与版本信息。

5.4 工程目录结构建议

以下目录仅展示推荐骨架，具体实现文件可按联调节奏逐步细化。

```
project/
├── configs/{step_graph,actions,prompts,workspace}/
├── src/{domain,adapters,perception,orchestration,execution}/
├── src/app.py
├── artifacts/  logs/
└── tests/{replay,mock}/
```

6. 运行观测、日志与回放

考虑到本项目必须在有限时间内完成联调与录制，回放能力应被视为一等能力。建议所有关键环节都输出 JSONL 事件，并与图像、动作结果、配置版本一起落盘。

6.1 建议记录的最小事件集

事件类型	关键字段	目的
frame_captured	run_id, frame_id, ts, camera_id, artifact_paths	定位相机采集与素材。
scene_analyzed	step_candidates, observed_facts, quality_score	分析视觉是否判断正确。
decision_made	decision, step_id, action_id, reason_code	分析控制器是否决策合理。
action_executed	action_id, status, error_code, telemetry	分析动作执行是否正常。
postcheck_done	expected_postconditions, actual_facts, result	分析动作是否真正达成目标。
run_finished	status, elapsed_time, final_step_id	收口本次任务结果。

6.2 素材归档建议

```
artifacts/
└── run_20260410_001/
    ├── frames/
    │   ├── frame_000123_rgb.jpg
    │   └── frame_000123_depth.png
    ├── events/
    │   └── runtime.jsonl
    ├── decisions/
    │   └── decision_trace.json
    └── configs_snapshot/
        ├── step_graph.yaml
        └── action_registry.yaml
```

6.3 回放使用场景

- 离线分析视觉误判与阈值设置是否合理。
- 复盘某一步骤为何进入 recover / fail。
- 录制前回看关键路径，确认每一步的口播说明与现场演示一致。

- 在不连真机的情况下，用 mock executor 重放控制链路。

7. 团队职责与协作机制

7.1 团队职责分工

成员	角色	主责模块	主要交付物
则轩	AI 算法工程师	PerceptionService、Orchestrator、StepGraph 规则、知识库与整体调度	视觉判断 API、顶层控制脚本、结构化输出规范
国强	SDK 工程师	ActionExecutor、机械臂与灵巧手原子动作序列、动作注册表	标准化动作接口、动作参数模板、执行错误码
宇渊	应用开发工程师	CameraAdapter、相机标定、ROI 配置、工作空间配置与动作配置协同	图像获取模块、标定配置、工作台参数文件

7.2 协作要求

- 每日 17:00 进行一次固定进度同步，按模块汇报当日可演示增量。
- 接口字段、配置项或错误码发生变化时，必须提前通知并同步更新文档与代码。
- 代码由顶层控制负责统一集成，联调期间禁止未同步的接口私改。
- 所有阻断问题统一归档到问题清单，标明影响模块、复现步骤、当前结论与处理人。

8. 开发计划、测试与验收

8.1 里程碑计划

阶段	截止日期	目标	主责
M1	2026-04-15	硬件集成完成，相机与 SDK 基础调用打通	宇渊
M2	2026-04-17	原子动作序列打包完成，单步手动调用验证通过	国强
M3	2026-04-19	视觉模型 + 知识库实现步骤识别准确率 > 90%	则轩
M4	2026-04-20	端到端 Demo 完整闭环，输出可演示版本	全员联调

8.2 测试策略

测试层级	测试内容	目标
单元测试	数据结构校验、枚举合法性、配置文件解析	保证基础接口稳定。
Mock 闭环测试	使用历史 FramePacket + mock executor 跑 StepGraph	提前发现控制逻辑错误。
单步联调测试	逐个验证动作与后置条件	保证每个 atomic action 可单独调用。
端到端测试	完整跑通所有步骤	保证 Demo 可演示。
稳定性测试	连续运行、多次重复、异常恢复	保证录制前可靠性。

8.3 验收清单

- 能够从纸箱放置开始自动进入 WAIT_BOX → OBSERVE → DECIDE → EXECUTE → VERIFY 闭环。
- 关键步骤识别输出为结构化结果，字段与枚举符合本文定义。
- 执行接口接收完整 ExecutionRequest，并返回完整 ExecutionResult。
- 日志中至少可追溯 frame_id、step_id、action_id、decision、error_code 与素材路径。
- 发生识别低置信、执行失败或后置条件不满足时，系统能够进入预设恢复逻辑。
- 项目目录、配置文件与文档一致，便于后续维护与复用。

9. 风险与控制

风险	影响	控制措施
视觉误判导致错误推进	可能执行不合适动作	强制后置条件验证；设置识别阈值；引入 retake 策略。
相机视角/标定漂移	事实抽取不稳定	固定工位与 ROI；记录标定版本；联调前重新核验。
动作执行完成但纸箱未折到位	状态与真实世界不一致	执行结果与步骤结果分离；必须复拍验证。
接口字段频繁变化	多人联调阻塞	冻结核心数据结构；统一枚举和 ID 命名；变更同步。
录制前无法快速定位问题	影响最终演示	强制落盘事件与素材；保留回放路径与配置快照。

工程落地建议

- 优先冻结 src/domain/models.py 与 configs/step_graph/carton_demo_v1.yaml 两个核心文件。
- 先用 mock executor 跑通完整控制闭环，再接真机 SDK 与真视觉，尽量将结构问题和硬件问题拆开定位。
- 固定工位条件下优先做 ROI / mask / 背景裁剪，再送视觉模型，可显著提升判断稳定性。

附录 A. 推荐的标准 ID 清单

以下 ID 建议作为第一版冻结清单，便于视觉、控制、执行三侧并行开发。

A.1 Step ID

```
S00_wait_box
S10_open_box_body
S20_fold_left_flap
S30_fold_right_flap
S40_fold_front_flap
S50_fold_rear_flap
S99_done
```

A.2 Action ID

```
A00_home_safe_v1
A10_open_box_body_v1
A20_fold_left_flap_v1
A30_fold_right_flap_v1
A40_fold_front_flap_v1
A50_fold_rear_flap_v1
```

A.3 Fact Key

```
box_present
body_opened
left_flap
right_flap
front_flap
rear_flap
box_pose_stable
task_done
```

附录 B. Mermaid 源码（便于后续维护）

如需在代码仓中维护图示，可直接使用以下 Mermaid 源码生成同类架构图。

B.1 总体架构图

```
flowchart LR
    Operator["操作员放置纸箱"]
    Cam["CameraAdapter\n采集 RGB-D"]
    Per["PerceptionService\n步骤/事实判定"]
    Ctl["Orchestrator\n状态机与调度"]
    Plan["StepGraph Planner\n步骤转移/恢复策略"]
    Exe["ActionExecutor\n原子动作执行"]
    SDK["Arm/Hand SDK"]
    Robot["双机械臂 + 灵巧手"]
    Store["Config / Log / Replay"]

    Operator --> Cam
    Cam --> Per
    Per --> Ctl
    Ctl --> Plan
    Plan --> Exe
    Exe --> SDK --> Robot
    Robot --> Cam

    Store --> Per
    Store --> Plan
    Store --> Exe
    Per --> Store
    Ctl --> Store
    Exe --> Store
```

B.2 运行状态机图

```
stateDiagram-v2
    [*] --> INIT
    INIT --> WAIT_BOX: 初始化完成
    WAIT_BOX --> OBSERVE: 检测到纸箱
    OBSERVE --> DECIDE: 输出 Observation
    DECIDE --> EXECUTE: 生成 action_id
    EXECUTE --> VERIFY: 动作执行结束
    VERIFY --> DECIDE: 后置条件满足且进入下一步
    VERIFY --> OBSERVE: 未满足后置条件但可继续观察
    VERIFY --> SUCCESS: 满足完成条件
    VERIFY --> RECOVER: 超时 / 低置信 / SDK 失败
    RECOVER --> OBSERVE: 重试 / 重抓 / 回退
    RECOVER --> FAIL: 超过重试上限
    SUCCESS --> [*]
    FAIL --> [*]
```